STEP-BY-STEP GUIDE

Build and Optimize an SEO-Ready Website with Claude Code

What you do, what you provide, and what Claude does at every stage, for any business that wants to rank on Google.

Structure drawn from Jono Catliff's "Claude Code SEO: How I Got 50,000 Clicks Per Month" (youtu.be/4IyJm1i__ag). Prompts and templates referenced here live in `docs/seo-kit/`. Hosting is on Render.

How to use this guide

The video's method is: build a fast, crawlable site → find keywords you can actually win → write posts and service pages in your own voice → apply an SEO checklist → fix technical issues until Lighthouse is green → deploy → tell Google. You do each stage **by hand once** so you can judge the output, then Claude's **skills** repeat it for every future page.

Each phase has four rows:

Row	Meaning
YOU PROVIDE	Information, files or decisions only you can supply (facts about the business, samples of your writing, exports, approvals).
YOU DO	Actions you take yourself: signing up, exporting, clicking, pasting a prompt.
CLAUDE DOES	What Claude Code carries out once you have given it the above.
DONE WHEN	A checkpoint. Do not move on until it is true.

Phase map

#	Phase	Video time	Core output
0	Accounts and tools	intro	Logins and keys ready
1	Business intake	context	Filled intake sheet
2	Project folder and CLAUDE.md	3:47	Instruction file + .env
3	Design reference and starter site	3:47	Home, blog index, service index
4	Confirm static generation	8:06	Pre-rendered HTML pages
5	Keyword research	10:06	keywords.csv
6	Your voice and real facts	22:18	references/ files
7	First blog post	18:00 – 28:52	One post, in your voice
8	On-page SEO pass	40:22	Checklist-verified post
9	Service pages	31:22	service-keywords.csv + first page
10	Technical SEO	45:52	sitemap, robots, Lighthouse 100
11	Use the skills	51:02	blog and service-page skills
12	Deploy on Render	1:01:11	Live website
13	Tell Google	1:03:24	Business Profile, Search Console
14	Publishing routine	53:16	Safe cadence
15	Improve conversion	31:22 + end	Winning landing page
16	Off-page (optional)	54:36	Safe backlinks only

NOTE Do not skip ahead. The video's warning: a wrong strategy only shows up 2–4 years later. Every checkpoint exists to catch problems while they are cheap to fix.

0 Accounts and tools

Set up once, before anything else

YOU DO	• Install **Claude Code on your own computer** (recommended: your files persist between sessions, the key stays on your machine, and local preview and Lighthouse work directly) and confirm you can start a session in a terminal inside a folder. No Antigravity or VS Code needed. • Install **Node.js** (needed to build the Next.js site). • Create a **Semrush** account (free trial) for keyword data. • **Decide how you will get images. You can use either way, per run, and switch any time.** (A) **Unsplash:** create a free Unsplash developer account, open Your apps → New Application, accept the terms and copy the **Access Key** (never the Secret Key). New apps start in demo mode: 50 requests per hour, plenty for one post at a time. (B) **Your own images:** no account needed; you will put files in an `images/` folder in Phase 2. • Create a **GitHub** account. • Create a **Render** account and connect it to GitHub. • Use one **Google** account for Business Profile, Search Console and Analytics. • Optional: a domain from Namecheap, or add one later in Render.
YOU PROVIDE	• **If using Unsplash:** your Access Key, copied somewhere safe, and the **name of your Unsplash app** (it is used in the credit links; the name may not contain the word "Unsplash"). You will paste the key into the `.env` file yourself in Phase 2. **Do not paste it into the chat.** • **If using your own images:** the photos you want on the site (team, work, premises, products). You must have the rights to use them.
DONE WHEN	• Every login works. If using Unsplash, the Access Key is saved somewhere safe. If using your own images, the files are gathered.

1 Business intake

Everything Claude needs to know about the business

YOU PROVIDE	• The completed **Intake Sheet** (Appendix A): business name, services, locations, contact details, target customer, goal of the site, and brand colors/logo if any. • Any **real facts** you want used: years in business, jobs completed, certifications, guarantees.
YOU DO	• Fill in Appendix A. Anything you do not know yet, write unknown (Claude will leave a TODO and not guess).
CLAUDE DOES	• Turns your answers into the project's `CLAUDE.md` (from `docs/seo-kit/CLAUDE.md.template`).
DONE WHEN	• `CLAUDE.md` contains no leftover `{{placeholders}}`.

2 Project folder and CLAUDE.md

Video 3:47 – “the instruction manual, like training an employee”

YOU DO	• Create an **empty folder** for the project and open Claude Code in it. • Put the completed `CLAUDE.md` in the folder root. • **If using Unsplash:** create the `.env` file in the project root (see the box below) and paste your Access Key into it. • **If using your own images:** create a folder named `images` in the project root and drop your files in (JPG, PNG, WebP or AVIF; subfolders such as `images/team` are fine). Claude never changes these originals. • Copy the kit’s `references/` templates and `on-page-seo-checklist.md` into the project.
CLAUDE DOES	• Creates `.gitignore` (with `.env` in it) and the folder layout: `content/blog`, `content/services`, `content/published.json`.
DONE WHEN	• Folder contains CLAUDE.md, references/, checklist, plus .env (Unsplash) and/or images/ (own images). If using Unsplash, Claude has confirmed that .env is git-ignored and UNSPLASH_ACCESS_KEY is set, **without printing the key**.

NOTE Unsplash key: where it goes and how Claude finds it. (1) Open your project folder, the same one where you run Claude Code and where CLAUDE.md lives. (2) Create a plain text file named exactly `.env`: leading dot, no .txt ending. (3) Put the one line below in it: no spaces around =, no quotes. (4) Do **not** put it in a subfolder, and never add `NEXT_PUBLIC_` to the name (that would publish the key to every visitor). (5) Ask Claude: "Check that .env is git-ignored and UNSPLASH_ACCESS_KEY is set. Do not print the key." Only the Access Key is needed. Never use the Secret Key.

```
my-site/
  CLAUDE.md
  .env  <-- Unsplash key goes here (project root)
  images/  <-- your own images go here
  .gitignore  (must list .env)
  references/
  content/

Inside .env, exactly one line:
UNSPLASH_ACCESS_KEY=paste_your_access_key_here
```

TIP You are only prompted once, here. After this, each run checks what it needs first: with Unsplash, that the key is in .env; with your own images, that images/ has files. If not, Claude stops and tells you how to fix it. Nothing is written until it is fixed. The key is used only while writing a post. The live site never needs it, and Render needs no secret for it.

TIP Choosing the image source each run. Tell Claude with the command: `blog images=unsplash` (pull from the Unsplash API), `blog images=local` (pick from your images/ folder), `blog images=none` (placeholders), or `blog images=mixed` (your images first, Unsplash to fill gaps). Plain words work too: "write the next post, use the photos I uploaded". A default can be set in CLAUDE.md; your instruction on a run always wins. If nothing is set, Claude asks once.

3 Design reference and starter site

Video 3:47 – build three pages only

YOU PROVIDE	• **A design reference.** Either one screenshot of a landing page whose look you like (free on Dribbble: search “[your industry] website”), **or a written design system** (colors, type, spacing, components), as in the Capstone build spec. Without a reference the result looks like generic “AI slop”. Also say **how many pages** you need and list them; the three starter pages in this guide are only a minimum. • Brand colors, logo and photos if you have them.
YOU DO	• Give Claude the screenshot (drag it in or give its path) and run **Prompt #1** from the kit.
CLAUDE DOES	• Builds the **homepage**, a **blog index** (lists all posts) and a **service index**, matching the design, and starts a local preview link.
DONE WHEN	• You opened the localhost link and approve the look and feel on all three pages, including on a phone-width window.

4 Confirm static generation

Video 8:06 – the pizza analogy

YOU DO	• Ask Claude: “Confirm this site is statically generated, run the build, and list the HTML files created.”
CLAUDE DOES	• Confirms static output is on (Next.js: `output: 'export'`, folder `out/`; Astro: static output, folder `dist/`), runs the build, and shows the generated folder with one .html file per page.
DONE WHEN	• Build passes and each page exists as ready-made HTML. (Why: Google must be able to crawl a finished page instantly. Server-built or browser-built pages risk not being ranked at all.)

TIP Render fit: a static export is hosted as a **Render Static Site** with build command `npm install && npm run build` and publish directory `out`.

5 Keyword research

Video 10:06 – “not every keyword is created equal”

YOU PROVIDE	• One or two **root keywords** for your business (e.g. the main thing you sell or do). • Your judgment on which keywords are customers and which are competitors.
YOU DO	• In Semrush open **SEO → Keyword Magic Tool**, enter the root keyword. • Filter: **Keyword Difficulty ≤ 30**, **Volume ≥ 100**, **Intent: informational** (for blog posts). • Hand-pick. Skip other companies’ brand names, “how to become a [profession]” queries, and irrelevant terms. Keep terms a future customer would search. • Also use the **Questions** tab, **adjacent / upstream topics** (same audience earlier in the buying journey), and **competitor domains** (see what they rank for). • Add picks to a Keyword List (aim for a few hundred to a few thousand over time). **Export as CSV**, rename it `keywords.csv`, and put it in the project. • Run **Prompt #2** so Claude summarizes the file and flags suspect rows.
CLAUDE DOES	• Reads the CSV, reports columns and row count, ranks the best rows, flags brand names and “become a” queries.
DONE WHEN	• `keywords.csv` is in the project and you have reviewed it.

6 Your voice and real facts

Video 22:18 – inject voice, humor, stats and stories

YOU PROVIDE	• **Writing samples**: LinkedIn posts, emails, transcripts of calls or videos, or passages online whose tone you love. • **Real numbers** (years, jobs done, typical prices and timeframes, review rating). • **Real stories** and things you always tell clients; your opinions on your industry; how funny you want to be. • **Permission** to share any story, anonymized by default.
YOU DO	• Run **Prompt #4** with the samples pasted in. • Open each file in `references/` (voice, tone, humor, opinions, stats, stories) and correct anything wrong.
CLAUDE DOES	• Fills the six files only from what you provided. It must not invent facts; missing items stay as placeholders and are listed for you.
DONE WHEN	• No placeholders remain in stats and stories; everything in them is true.

TIP The video’s humor rule: if the first 50 words contain no dad joke, self-aware aside or vivid image, rewrite them. Adjust the persona to your brand, and dial humor down for serious topics.

7 First blog post

Video 18:00 – 28:52 – draft, add your voice, steal the winning format

YOU PROVIDE	• Choose **one keyword** from `keywords.csv`. • Answers to any `TODO(owner)` lines Claude leaves.
YOU DO	• Run **Prompt #3** (first draft with keyword cluster and images; say which image source to use). Read it. The video calls this draft "crap" on purpose; it is not meant to be final. • Run **Prompt #5** (rewrite). • Compare the old and new opening side by side and approve or request changes.
CLAUDE DOES	• **Draft:** first checks the image source you chose (the Unsplash key in `.env`, or files in `images/`) and stops and asks you if it is not ready, then writes the post, builds a **keyword cluster** (related terms the same page can rank for), and adds images. **Unsplash:** hotlinks the photos with the required credit and shows you its picks. **Your own images:** looks at them, picks the ones that fit, and saves optimized copies (originals untouched). • **Rewrite:** searches Google for the keyword, analyzes the **top 3 organic results** (skipping Reddit and forums), records word count, H2s and images, **averages** them as your targets, then rewrites in your voice, humor, opinions, stats and stories.
DONE WHEN	• The post sounds like you, hits the averaged length/structure, and nothing in it is invented.

8 On-page SEO pass

Video 40:22 – "80+ signals in one prompt"

YOU DO	• Run **Prompt #6** on the post (the kit checklist has 90 items). • Spot-check three things: internal and external links are real and relevant; the keyword is in the first 100 words; the **meta title and description** exist (Chrome → right-click → View Page Source, or DevTools → Elements, search "og").
CLAUDE DOES	• Applies the checklist **while keeping the humor**: one H1, 3–5 internal links, 2–3 external links, 4–8 FAQ questions, meta title and description, Open Graph tags, schema markup, alt text. Prints an audit report and fixes failures.
DONE WHEN	• Audit report shows no failed items (or you accepted the exceptions) and the post is still fun to read.

NOTE Not every site has service pages. A charity, membership body or portfolio has programs or pages with fixed copy, so skip this phase and set `service_pages: NOT USED` in CLAUDE.md. The blog does the SEO work instead.

9 Service pages

Video 31:22 – the zipper method

YOU PROVIDE	• Your list of **services** and your list of **locations** (cities or municipalities). • Proof for the page: reviews, licences, guarantees, process, price guidance if you give any. • Which pages matter most. Be selective: the video says never thousands and probably not hundreds.
YOU DO	• In Semrush, Keyword Magic Tool, enter your root keyword and **sort by CPC** (cost per click). Typical real CPC is about $3–$20; ignore absurd outliers. High CPC marks money keywords. • Look for the pattern service + city. Add chosen terms to a second list, **export as CSV**, rename it `service-keywords.csv`. • Run **Prompt #7** for one page (say which image source to use), then run the on-page pass (Prompt #6) on it.
CLAUDE DOES	• Builds a landing page for that service in that city using the homepage's conversion layout, with unique local copy (not a swapped city name), a form above the fold, FAQ, and links to related blog posts.
DONE WHEN	• First service page approved; keyword recorded in `content/published.json`.

TIP Zipper example: services (emergency plumber, drain cleaning) × cities (Toronto, Vancouver) = emergency plumber Toronto, drain cleaning Vancouver, and so on. Blogs raise your whole domain's trust ("a rising tide lifts all boats"); service pages are the boats that earn.

10 Technical SEO

Video 45:52 – sitemap, robots.txt, Lighthouse 100

YOU DO	• Open the site in Chrome → three dots → More tools → Developer tools → **Lighthouse** tab → Analyze page load. • Click to **expand every failing section** (Performance, Accessibility, Best Practices, SEO), then **copy the whole report**. • Run **Prompt #8** with the report pasted in. • Re-run Lighthouse. If anything is below 100, paste the new report and repeat. (The video stopped happily at 100 / 100 / 100 / 96.)
CLAUDE DOES	• Fixes render-blocking resources, heavy JavaScript, image sizing, contrast and other issues; creates `sitemap.xml` (every public page) and `robots.txt` (allow everything except private pages like admin, and point to the sitemap).
DONE WHEN	• Lighthouse is at or near 100 on all four scores; `/sitemap.xml` and `/robots.txt` open in the browser.

TIP Re-test on the live Render site after deploy, and test both mobile and desktop. Scores vary between runs.

11 Use the skills

Video 51:02 – “bottle it into a reusable Claude Code skill”

YOU PROVIDE	• Phases 1–10 done by hand once: a post and a service page you are happy with. The skills repeat that exact process, so they are only as good as the references, CLAUDE.md and keyword files you gave them. • For each run: **which image source** to use (Unsplash, your own images, mixed or none).
CLAUDE DOES	• The **blog** and **service-page** skills are already installed in `.claude/skills/`. Each one checks its prerequisites first, picks the next unused keyword, researches the top 3 results, writes in your voice, adds images from the source you chose, runs the checklist, publishes through the standard template and updates `content/published.json`.
YOU DO	• Start a **new chat** so the skills load, then type: `blog`, `blog images=local`, `blog images=unsplash`, `service`, or `service <service> <city>`. • Review the report: keyword, targets vs actuals, the images used (with links for Unsplash), every `TODO(owner)`. • Optional: schedule it (the video’s example is a daily run at 9:00 a.m.). Only after the cadence rules in Phase 14.
DONE WHEN	• Typing **blog** produces a new, unique, voice-matched, checklist-passing post using the next unused keyword and the image source you chose.

12 Deploy on Render

Video 1:01:11 (Vercel in the video; Render here)

YOU DO	• On GitHub, create a **new private repository**. • Run **Prompt #10**: Claude commits and pushes the project (never `.env`). • In Render: **New → Static Site** → connect GitHub → pick the repo and branch. • Set **Build Command** to `npm install && npm run build` and **Publish Directory** to `out` (Next.js static export) or `dist` (Astro). Create the site. • Optional: Settings → **Custom Domains**; add the DNS records Render shows at your registrar.
YOU PROVIDE	• The GitHub repo URL; your domain name if you have one. (Your `.env` is not uploaded and not needed on Render. Unsplash photos load from Unsplash’s servers, as their rules require; your own images are deployed with the site.)
CLAUDE DOES	• Commits and pushes. If a build fails on Render, paste the log and Claude fixes it. Every future push to the branch redeploys automatically.
DONE WHEN	• The live Render URL opens every page, `/sitemap.xml` and `/robots.txt` work, and a bad URL shows your 404 page.

NOTE Only when the real domain is live. If the site is on a temporary address with indexing switched off (for example `PUBLIC_INDEXABLE=false`, which adds noindex to every page), do **not** do this phase yet. Wait until the domain is connected and indexing is on. Values like these are baked in at build time, so **redeploy** after changing one.

13 Tell Google

Video 1:03:24 – four steps after launch

YOU DO	• **Google Business Profile** (called Google My Business in the video): create the free listing. The video calls it the lowest-hanging fruit for clicks. Keep name, address and phone identical to your site. Best for local, service-area businesses. • **Search Console**: add your live URL as a **URL prefix** (full URL including https). Choose **HTML tag** verification and run **Prompt #11** with the tag. Wait for Render to redeploy, then click Verify. • In Search Console → **Sitemaps**, submit `sitemap.xml`. • For each new page: paste its URL into the top bar → **Request indexing** (limit roughly 10 a day). Can put a page on Google within about a day instead of weeks. • **Google Analytics**: set up a property so visitor behavior is tracked.
YOU PROVIDE	• The verification meta tag from Search Console; the Analytics measurement ID.
CLAUDE DOES	• Adds the tag/script, commits, and redeploys.
DONE WHEN	• Search Console shows the property verified and the sitemap “Success”; Analytics records your own visit.

14 Publishing routine and cadence

Video 53:16 – “don’t dump 1,000 posts at once”

YOU DO	• Follow the ramp below. Review each post before it goes live. • Each run: type **blog** (or the service-page skill), review the audit report, answer any TODO(owner), deploy, request indexing.
YOU PROVIDE	• New real stats and stories as they happen; approval before each deploy.
CLAUDE DOES	• Picks an unused keyword, researches the top 3, writes, optimizes, records the keyword so it is never reused, and stops at the cadence limit.
DONE WHEN	• Posting is steady and never spikes.

Day	Posts that day
1	1
2	1
3	2 (maybe)

Day	Posts that day
5	3
8	4
After	Keep it reasonable. Never anything like 1,000 a day.

15 Improve conversion

Video 31:22 and closing – “your business is a funnel”

YOU DO	• Treat each stage separately: visitors → leads (form) → customers (closed deals). • Make 5–10 landing-page variations and test them (the video used Google Ads to send traffic). His best page converted about 20%. • Roll the **winning layout** out to all service pages.
YOU PROVIDE	• Conversion results; which variation won.
CLAUDE DOES	• Builds variants, applies the winner across the service pages, rebuilds and redeploys.
DONE WHEN	• All service pages share the best-converting layout.

16 Off-page SEO (optional)

Video 54:36 – the riskiest part

YOU DO	• **Avoid** cheap backlink packages (“100 links for $5”). These are usually private blog networks and can get a site permanently penalized. Ask any provider for before-and-after proof. • Safe methods the video lists: **broken-link swapping** (find a competitor’s dead outbound links, offer your page as the replacement); **guest posting** (search your niche + “write for us”, include one link back); **journalist queries** (answer expert questions for a credited link); **paid quality placements** on reputable sites, handled carefully.
CLAUDE DOES	• Drafts outreach emails and pitches (Prompt #12) in your voice.
DONE WHEN	• Any backlink work is slow, relevant and verifiable. The video’s own advice: ranking on well-chosen keywords matters more than this.

Appendix A – Intake sheet

Fill this in before Phase 1. Write unknown where you do not know; Claude will leave a TODO and not guess.

Section	Question	Your answer
Business	Business name	
Business	One sentence: what you do and for whom	
Business	Years in business / founded	
Business	Website goal (calls, forms, bookings, sales)	
Business	Business type (local service, online service, nonprofit, shop, etc.)	
Offer	Services (list every one you want a page for)	
Offer	Starting prices or price ranges (only if you will publish them)	
Offer	Guarantees, licences, certifications	
Places	Cities / areas served (or “online”)	
Places	Physical address, if customers visit	
Contact	Phone, email, hours (exactly as they should appear everywhere)	
Customer	Who your ideal customer is; what worries them	
Customer	Top 10 questions customers ask you	
Brand	Logo, colors, fonts, photos available?	
Brand	Websites whose design you like (for the Dribbble-style reference)	
Voice	How funny / formal should the writing be?	
Voice	Words or topics to never use	
Proof	Review count and rating; best testimonials (with permission)	
Proof	Real stats: jobs done, response time, results	
Proof	2–3 true stories from real work (anonymized)	
Competition	3 competitors or sites you want to outrank	
Setup	Domain name (own it already?)	
Setup	GitHub username; Render account ready?	
Setup	Google account for Business Profile / Search Console / Analytics	

Appendix B – Numbers to remember

Item	Target (from the video unless noted)
Keyword difficulty	30 or below (about 2/10 can reach the top 3 fast)
Monthly search volume	100 or more
Blog keyword intent	Informational; service pages use money keywords sorted by CPC (real CPC about $3–$20)
Keywords per page	A primary keyword plus a cluster of related terms
Competitor benchmark	Top 3 organic results; average their word count, H2s and images
Opening hook	Joke / aside / vivid image inside the first 50 words
Primary keyword placement	First 100 words
Internal / external links	3–5 internal, 2–3 external per blog post
FAQ questions	4–8 per blog post
H1 tags	Exactly one per page
Lighthouse	100 on SEO, Accessibility, Best Practices, Performance
Service pages	Selective; not thousands, probably not hundreds
Indexing requests	About 10 a day in Search Console
Publishing ramp	1, 1, 2, 3, 4 per day, then steady
Best landing-page conversion (his)	About 20%
Click share (cited)	About 40% on result #1, 20% on #2

Appendix C – Per-run checklist (after the skills are installed)

- ☐ Keyword chosen from the CSV is unused and still fits the business.
- ☐ Top-3 research table looks sensible (right competitors, no forums).
- ☐ First 50 words have the hook; no generic AI opening.
- ☐ Every stat and story is real; every TODO(owner) answered.
- ☐ Audit report: no failed items; voice check passed.
- ☐ `npm run build` passes; Lighthouse still near 100.
- ☐ Deployed on Render; page opens live.
- ☐ Request indexing in Search Console (stay under ~10 a day).
- ☐ Cadence limit for today respected.

Appendix D – Troubleshooting

Problem	What to do
Claude says the Unsplash key is missing	Check .env is in the project root (same folder as CLAUDE.md), named exactly .env, one line: UNSPLASH_ACCESS_KEY=your_access_key, no spaces or quotes. Restart the Claude session and retry. If the key was ever pasted into a chat or committed, regenerate it in your Unsplash app settings.
Unsplash says rate limit reached	Demo apps get 50 requests per hour. Wait for the hour to reset, use `images=local` for now, or apply for production access in your Unsplash app.
Claude says images/ is empty or has no good match	Add more images to images/ (subfolders help), or choose `images=mixed` so Unsplash fills the gaps, or `images=none` for placeholders.
Writing sounds generic	Add more real samples to references/; re-run Prompt #4 and #5.
Lighthouse stuck below 100	Paste the new full report with failing sections expanded; repeat Prompt #8.
Render build fails	Paste the build log to Claude. Confirm static output is on and the publish directory matches the framework (`out` for Next.js, `dist` for Astro).
Search Console will not verify	Make sure the tag is on the deployed site (view page source), wait for the redeploy, then Verify.
Pages not appearing on Google	Confirm sitemap submitted, request indexing for key pages, check robots.txt is not blocking.
Post feels over-optimized or dry	Re-run the rewrite and tell Claude to restore the humor; content quality outranks checklist items.

Appendix E – Starting a project from a ready build spec (the Capstone flow)

When you already have a full build spec (like the Capstone Foundation one), the guide's phases 1–4 are replaced by a gated start. The spec is the source of truth for copy and design; this guide supplies the SEO method. Use `docs/START-PROMPT.md` in a new chat. **For a brand-new project with no spec**, the `site-setup` skill does the same job for any business: type `site-setup` in a new chat in an empty folder. It runs the intake, writes CLAUDE.md and the reference files only from your answers, creates the folders, proves the safety checks with a script, stops for your uploads, agrees a design direction and plan, and ends by writing a ready-to-paste build prompt (docs/BUILD-PROMPT.md). Type `site-setup audit` on an existing project to check it without changing anything.

Stage	What happens	You do
1 Folders	Claude checks Node, git and the skills, prints and creates the folder tree, makes .gitignore, .env and .env.example, then proves the ignores work.	Read the checklist it gives you.
(pause)	Claude stops.	Put the logos in public/brand/, approved photos in images-incoming/, and your Unsplash key in .env if used. Say "go".
2 Check uploads	Lists your files, flags receipts, IDs, screenshots and oversized or duplicate files. Changes nothing.	Say "go".
3 Plan	File tree, component list and questions, including your open decisions.	Answer and approve.
4 Build	Scaffold, design tokens, pages in the spec's order, image pipeline, SEO and structured data. A build and a 2–3 sentence update after each step.	Approve the slot map before photos are placed.
5 Acceptance	Checks each "Done means" item with proof (Lighthouse 95+, 360/768/1280 px).	Review.
6 SEO	Only when you ask: the blog skill writes draft stories in your voice, from keywords.csv.	Review each draft, then flip it to published.

TIP Never paste an API key into the chat. Type it into .env yourself. The skills stop and tell you if a key or the images folder is missing.